

SECP3133-01 · HIGH PERFORMANCE DATA PROCESSING

Optimizing High Performance Data Processing for Large Scale Web Crawlers

Lecturer: Assoc. Prof. Mohd Shahizan Bin Othman

Iman Abadi (A23CS0084) · Mohamed Alif Fathi (A23CS0112) · Dheshieghan (A23CS0072)

Background & Objectives

Target: BookXcess

Malaysian online bookstore with a large catalog across genres. Crawler parses XML sitemap and fetches product data via public `.json` endpoints.

Core Objectives

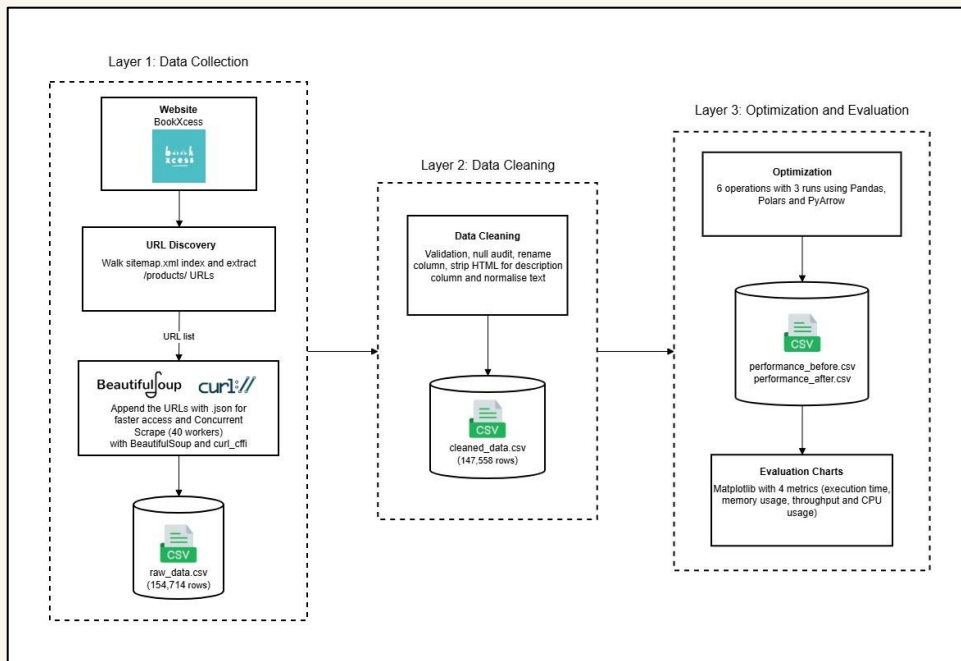
- Collect 100,000+ product records
- Clean and preprocess raw data with pandas
- Benchmark Pandas, PyArrow, and Polars

Dataset

raw_data.csv: 154,714 rows

cleaned_data.csv: 147,558 rows

Pipeline Architecture



Four notebooks execute the pipeline: `main_crawler.ipynb`, `clean_data.ipynb`, `optimize_pipeline.ipynb`, and `evaluation_charts.ipynb`.

Technology Stack



HTTP & Concurrency

`curl_cffi` mimics browser handshakes to bypass anti-bot protections. `ThreadPoolExecutor` with 40 workers handles I/O-bound crawling.



Optimization & Measurement

Polars and PyArrow benchmarked against Pandas. `psutil` tracks CPU and memory; `time.perf_counter` measures execution speed.



Parsing & Cleaning

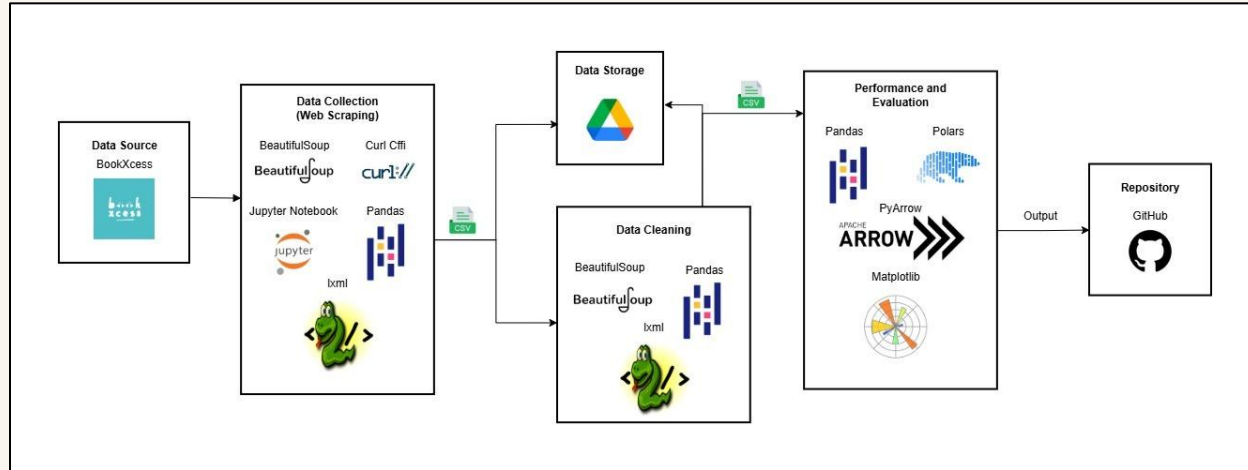
BeautifulSoup + lxml extracts HTML/XML. Pandas, `html`, and `unicodedata` organize and standardize data.



Visualization

Matplotlib transforms benchmarking data into comparative charts across four performance metrics.

Tools Used Diagram



Team & Responsibilities

Mohamed Alif Fathi — Group Leader & Data Engineer

Coordinated the team, built the crawler, and managed data ingestion and storage.

Iman Abadi — HPC & Performance Specialist

Designed optimization benchmarks and produced comparison charts.

Dheshieghan — Analyst & Documentation Lead

Performed data cleaning and analysis and guided the report and slides.

Crawling Strategy & Reliability

Two-Phase Crawl

- **Phase 1 — Discovery:** Walk `/sitemap.xml`, extract all `/products/` URLs, de-duplicate
- **Phase 2 — Scrape:** Append `.json` to URLs; fetch via 40-worker `ThreadPoolExecutor`

Reliability Measures

- **Per-request jitter:** 50–200 ms random delay
- **Retry logic:** Up to 3 retries with exponential backoff (1.5–3.5 s) for HTTP 429/500/502/503/504
- **Backup CSV:** Flush every 5,000 rows to prevent data loss
- **Failed URL recovery:** Re-scrape failures with half-sized worker pool

Cleaning & Transformation Steps

01

Drop Missing Values

Remove incomplete product records.

03

Date Formatting

Convert `created_at` to datetime.

05

Title & Text Normalization

Remove `[Bargain Corner]` prefix; decode HTML entities; normalize Unicode; collapse whitespace.

02

Column Standardization

Rename to lowercase with underscores; drop redundant `currency` column (all MYR).

04

HTML Cleaning

Strip tags from `body_html` → plain text `description`.

06

Validation & Price Logic

Validate author/publisher names; drop rows where `compare_price < current_price` or zero; calculate `discount_pct`.

Benchmark Operations

Six operations benchmarked across **Pandas**, **Polars**, and **PyArrow** — each run 3 times and averaged.

1. Filter High Discount

`discount_pct > 70 AND current_price < 20`

2. GroupBy Publisher

Mean price, mean discount, title count per publisher

3. String Extract

Regex extract "RM" price tier from tags

4. Calculate Savings

Derive `savings_rm` and `savings_pct_check`

5. Sort

By `discount_pct` (desc) and `current_price` (asc)

6. Top Authors

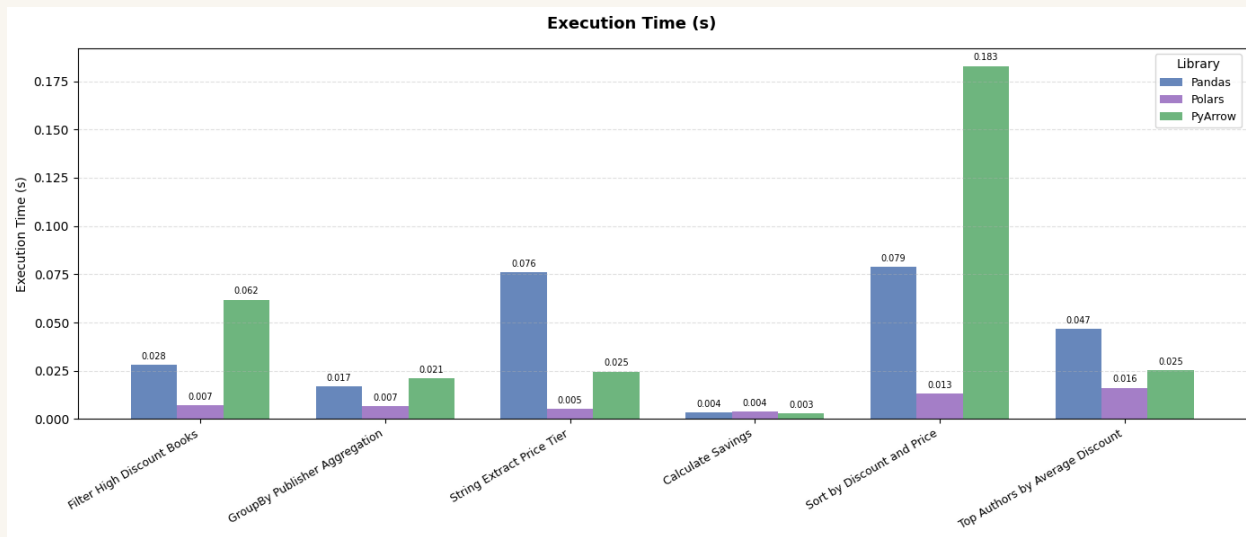
Top 20 authors by average discount

Benchmarking Results

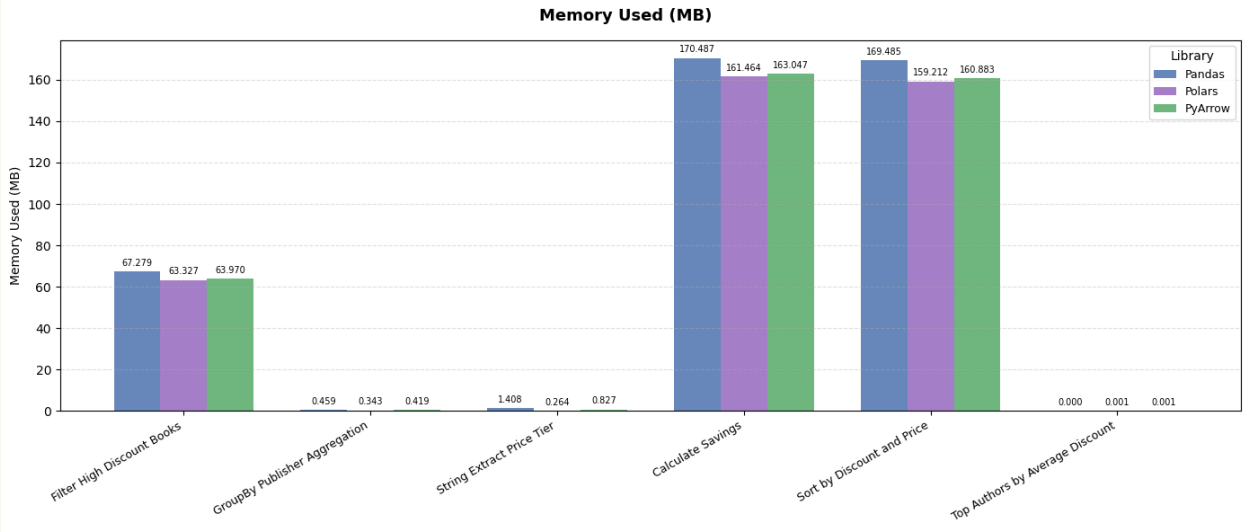
Key Findings

- **Polars** consistently fastest — Rust-based, columnar, multi-threaded
- **PyArrow** slowest on Sort (0.183 s) — nearly 2× Pandas
- **Throughput:** Polars peaks at 28.5M rows/s (String Extract)
- **Memory:** Polars & PyArrow slightly more efficient via columnar storage
- **CPU:** All under 18%; Polars most efficient at low usage

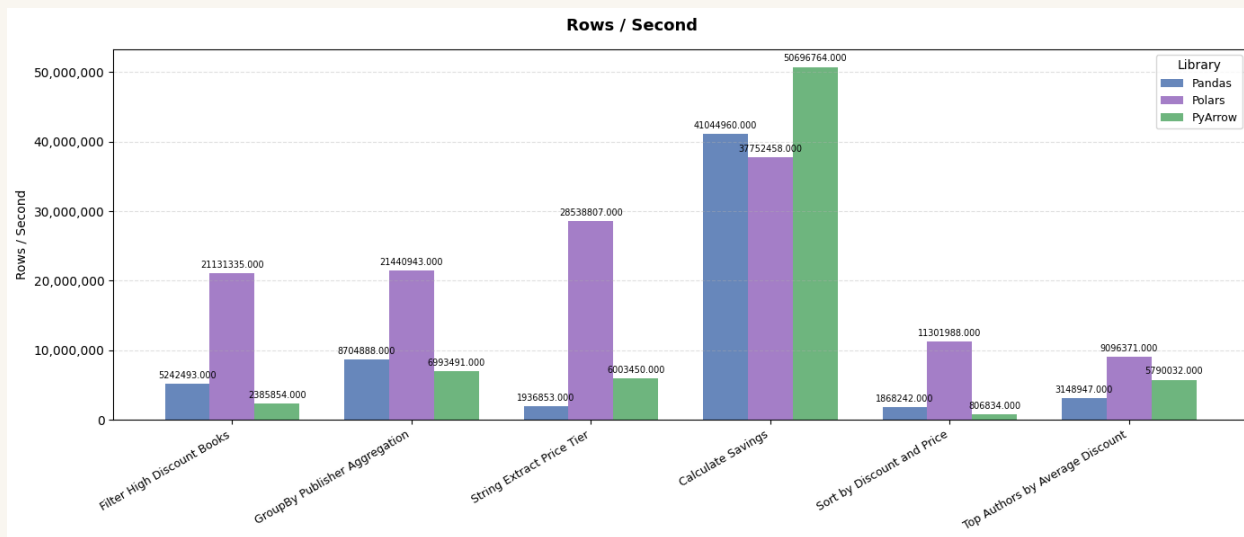
Execution Time



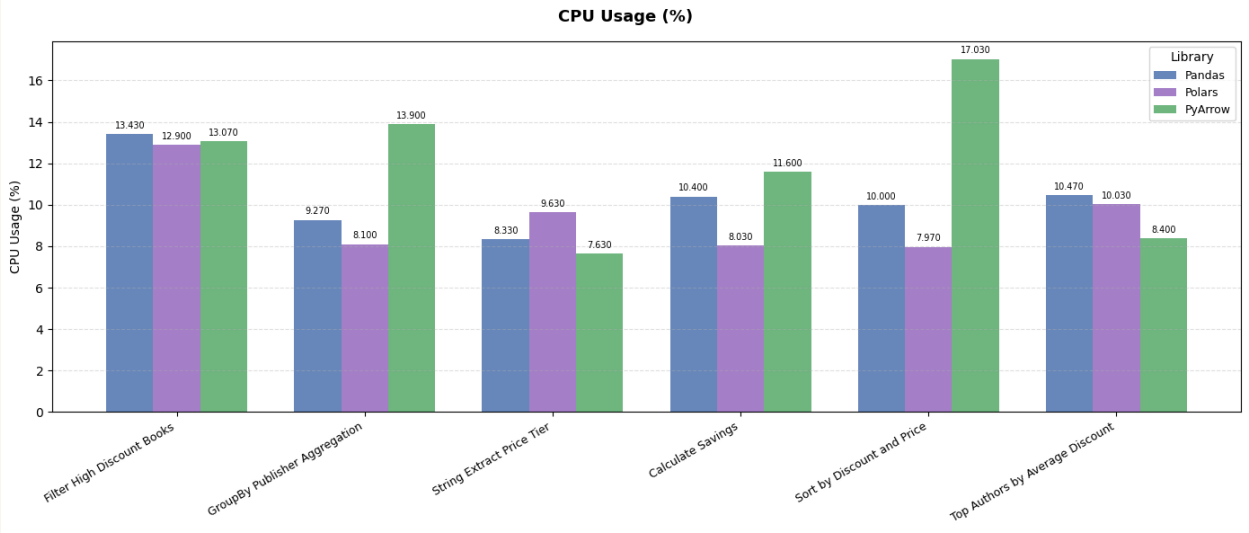
Memory Usage



Throughput



CPU Usage



Challenges, Findings & Future Work

Challenges

- Mislabeled Shopify fields ("Product Type" = author)
- Anti-bot & CAPTCHA blocking automation
- Noisy benchmarks required memory flush + 3-run averaging

Summary

154,714 → 147,558 clean records. **Polars** outperformed across all metrics.

Future Improvements

- Store in PostgreSQL / MongoDB
- Use Parquet instead of CSV
- Incremental crawling for new/updated products
- Benchmark on 1M+ records
- Build Power BI / Streamlit dashboard